Self Consistent Stochastic Interpolants

Clark Miyamoto (cm6627@nyu.edu)

March 22, 2026

Setup

Today we'll be working on a new setup. You are given:

- A dataset $\{y_i\}_i$ of experimental measurements $y_i \in \mathcal{Y}$ which correspond to an underlying signal $x_i \in \mathcal{X}$.
- Oracle access to a forward model $\mathcal{F}(x_i) = y_i$ which you believe accurately represents the data. So we're assuming no model misspecification. We'll denote $\text{Law}(y) = \mu$. The forward model implicitly defines a conditional probability $p(dy|x)$.

  The model can be stochastic (and thus non-invertible at the level of individual samples).

Your objective is to find x the signal when presented with a new y observation. These types of problems are called **inverse problems**; note this is different from doing a Bayesian inference as that requires stating a prior.

> Some examples:
>
> - **Reconstruction:** y is a CT scan, x is the 3D model of the tissue, $\mathcal{F}$ is the Radon transformation.
> - **Denoising:** y is photos from a camera, x is the "true" photos, $\mathcal{F}$ is a model of the lens (chromatic aberration, distortion) and CMOS sensor (electronic noise, sensor erosion).

Approach

High Level Idea The forward model $\mathcal{F}$ implicitly defines a map between the distributions $\mathcal{K} : \mathcal{P}(\mathcal{X}) \to \mathcal{P}(\mathcal{Y})$. I'll denote $\mathcal{K}$ as the integral operator on the densities.

$$\mathcal{K}\pi = \mu \tag{0.1}$$

Consider the inference time of a generative model Φ_Θ (where Θ is the neural network's parameters). This is the full integration of the ODE/SDE. If you have samples $y \sim \nu$, then applying $\Phi_\Theta(y) = X$ you have implicitly defined a pushforward on the distribution

$$(\Phi_\Theta)_\# \mu = \pi_\Theta \tag{0.2}$$

Let's assume that you've somehow trained the generative model to approximate $\pi_\Theta \approx \pi$. You can plug (0.2) into (0.1). This yields the **self-consistency equation**

$$\mathcal{K}\pi_\Theta = \boxed{\mathcal{K}(\Phi_\Theta)_\# \mu = \mu} \tag{0.3}$$

What is this saying? The properly trained generative model Φ_Θ should move your noisy observations μ to clean signals π_Θ, and then if I put these through the forward model $\mathcal{K}$, this should get you back to your noisy observations μ. We have stated how the generative model should behave.

Review of Stochastic Interpolants In a previous paper, this was implemented on diffusion models, but there's a more natural way, stochastic interpolants! So consider the stochastic interpolant

$$x_t = \alpha_t x_0 + \beta_t x_1 + \gamma_t z \tag{0.4}$$

where $\nu(x_0, x_1)$ are drawn from a coupled distribution $\int \nu(x_0, x_1)\, dx_1 = \pi$ and $\int \nu(x_0, x_1)\, dx_0 = \mu$. That is independent from $z \sim \mathcal{N}(0, \mathbb{I})$, and $\alpha_t, \beta_t, \gamma_t$ have the usual boundary conditions. At inference time, you can run the backwards SDE

$$dX_t = [\hat{b}_t(X_t) + \epsilon_t \gamma_t^{-1} \hat{\eta}_t(X_t)]\, dt + \sqrt{2\epsilon_t}\, dW_t \tag{0.5}$$

where ϵ_t is the noise scheduler. $\hat{b}_t$ and $\hat{\eta}$ are the drift & denoiser which can be learned using

$$\mathcal{L}_b(\hat{b}) = \int_0^1 \mathbb{E}[|\hat{b}_t(I_t) - \dot{I}_t|^2]\, dt \tag{0.6}$$

$$\mathcal{L}_g(\hat{\eta}) = \int_0^1 \mathbb{E}[|\hat{\eta}_t(I_t) - z|^2]\, dt \tag{0.7}$$

Upon learning, you have the SDE to equal in distribution to the interpolant $\mathrm{Law}(X_t) = \mathrm{Law}(x_t)$.

Enforcing the self-consistency equation In the original stochastic interpolants framework, you need access to the target distribution π and base distribution ν. But in our setup, you only have access to samples $y \sim \nu$. A potential interpolant which conserves this is

$$x_t = \alpha_t x + \beta_t \mathcal{F}(x) + \gamma_t z \tag{0.8}$$

Where $x \sim \pi$. Since we believe $\mathcal{F}(x) = y \sim \nu$, this has the correct boundary conditions: $x_{t=0} \sim \pi$ and $x_{t=1} \sim \nu$. However, we don't have access to $x \sim \pi$, we only have access to $y \sim \nu$.

Instead we can use the flow (induced by the backwards SDE) to conserve the interpolant

$$\boxed{x_t = \alpha_t \Phi_\Theta(y) + \beta_t \mathcal{F}(\Phi_\Theta(y)) + \gamma_t z} \tag{0.9}$$

Because Φ_Θ has to be initialized, you'll need to iteratively train your network to construct proper samples. The procedure goes as following:

$$\boxed{\Theta^{(k)} \xrightarrow[\text{'E' step}]{\text{via (0.9)}} I_f^{(k+1)} \xrightarrow[\text{'M' step}]{\text{minimizers in (0.6)(0.7) with } I_f^{(k+1)}} \Theta^{(k+1)}} \tag{0.10}$$

A couple comments:

- This is the same idea used in rectified flows. You train the neural network, then you use a coupling $\gamma(y, \Phi(y))$ to construct another flow, retrained the network, etc.

- The authors compare this to the EM algorithm (or other bi-level sch, where the E

Fact 1 (Uniqueness of Fixed Point) *Assume that $\mathcal{K}$ is injective, and the iterative scheme converges to a fixed point Θ^*. Then $\pi_{\Theta^*} = \pi$.*

Proof: Recall if $\mathcal{K}$ is injective, this means $\mathcal{K}\tilde{\pi} = \mathcal{K}\pi$ implies $\tilde{\pi} = \pi$.

Algorithm 1: Training of Self-Consistent Stochastic Interpolant (SCSI)

Input: Observation distribution μ, Forward mapping $\mathcal{F}$, Interpolant schedule $(\alpha_t, \beta_t, \gamma_t)$, Initialization of drift and denoiser $\Theta^0 = \{b^{(0)}, g^{(0)}\}$, Total number of iterations K, Number of transport steps T_{tr}

Output: Optimized networks $\Theta^{(K)} = \{b^{(K)}, g^{(K)}\}$

```
Θ ← Θ^(0) ;                                   // Initialize transport map
for k in 1...K do
    for i in 1...T_tr do
        y ~ μ ;
        x = Φ_{Θ^(k-1)}(y) ;                  // Backward transport to get a data sample
        ỹ = F(x) ;                            // Map back to observations
        z ~ N(0,1); t ~ U(0,1) ;
        I_t = α_t x + β_t ỹ + γ_t z ;
        SGD update of Θ via losses (3)(4) ;
    end
    Θ^(k) ← Θ ;                               // Update transport map
end
return Θ^(K)
```

Convergence of Algorithm: Example on AWGN on Gaussian

I want to verify if SCSI algorithm converges (and how fast) on a toy problem. In the AWGN on Gaussian, everything stays Gaussian at every iteration, so it makes a good test problem.

1 Setup

- **True data distribution:** $\pi = \mathcal{N}(b, \Sigma)$ with $\Sigma \succ 0$. This is unknown to the algorithm.
- **Forward model:** $\mathcal{F}(x) = x + w$ where $w \sim \mathcal{N}(0, \mathbb{I})$, independently of x. Meaning the observed distribution $\mu = \mathcal{N}(b, \Sigma + \mathbb{I})$.
- **Interpolant:** We use the linear interpolant

  $$I_t = (1 - \sqrt{t})\, x + \sqrt{t}\, \mathcal{F}(x) = x + \sqrt{t}\, w,$$

  where $x \sim \mathcal{N}(\tilde{b}, \tilde{\Sigma})$ is a generic Gaussian prior and $w \sim \mathcal{N}(0, \mathbb{I})$ is independent. Thus

  $$\text{Law}(I_t) = \mathcal{N}\left(\tilde{b},\ \tilde{\Sigma} + t\,\mathbb{I}\right).$$

- **Initial guess:** $\pi^{(0)} = \mathcal{N}(b_0, \Sigma_0)$ with $\Sigma_0 \succ 0$.

Because of the Gaussian structure, every iterate will be Gaussian: $\pi^{(k)} = \mathcal{N}(b_k, \Sigma_k)$.

Roadmap

1. Find the update rule: given $\pi^{(k)}$, what is $\pi^{(k+1)}$?
2. Compute the error $\|\pi - \pi^{(k)}\|$ across iterations.
3. Determine the convergence rate.

2 Step 1: Update Rule

2.1 Forward process (Heat semigroup)

Fix the current iterate $\pi^{(k)} = \mathcal{N}(b_k, \Sigma_k)$. The law $p_t = \mathrm{Law}(I_t)$ evolves under the heat equation

$$\partial_t \, p_t = \frac{1}{2} \, \Delta \, p_t, \qquad p_{t=0} = \pi^{(k)}, \tag{2.1}$$

with terminal condition $p_{t=1} = K\pi^{(k)} = \mathcal{N}(b_k, \Sigma_k + \mathbb{I})$.

2.2 Reverse process

Define the reverse-time variable $\tau = 1 - t$, and set $q_\tau := p_{1-\tau}$. Then

$$\partial_\tau \, q_\tau = - \, \partial_t \, p_t|_{t=1-\tau} = -\frac{1}{2} \, \Delta \, q_\tau.$$

We split the Laplacian using a diffusion coefficient $\epsilon \geq 0$:

$$\begin{aligned} \partial_\tau \, q_\tau &= -\frac{1+\epsilon}{2} \, \Delta \, q_\tau + \frac{\epsilon}{2} \, \Delta \, q_\tau \\ &= -\nabla \cdot \left[\frac{1+\epsilon}{2} \, (\nabla \log q_\tau) \, q_\tau \right] + \frac{\epsilon}{2} \, \Delta \, q_\tau. \end{aligned} \tag{2.2}$$

This is the Fokker-Planck equation associated with the reverse SDE

$$dY_\tau = \frac{1+\epsilon}{2} \, \nabla \log q_\tau(Y_\tau) \, d\tau + \sqrt{\epsilon} \, dW_\tau. \tag{2.3}$$

where $\mathrm{Law}(Y_\tau) = q_\tau$. Since $p_t = \mathcal{N}(b_k, \Sigma_k + t \, \mathbb{I})$, the score is $\nabla \log p_t(x) = -\,(\Sigma_k + t \, \mathbb{I})^{-1}(x - b_k)$. Substituting $q_\tau = p_{1-\tau}$ gives $\nabla \log q_\tau(x) = -\,(\Sigma_k + (1-\tau) \, \mathbb{I})^{-1}(x - b_k)$. Plugging this into (2.3) we get:

$$dY_\tau = -\frac{1+\epsilon}{2} \, (\Sigma_k + (1-\tau) \, \mathbb{I})^{-1}(Y_\tau - b_k) \, d\tau + \sqrt{\epsilon} \, dW_\tau. \tag{2.4}$$

We can now solve the SDE:

Define $A(\tau) := -\frac{1+\epsilon}{2} \, (\Sigma_k + (1-\tau) \, \mathbb{I})^{-1}$, so the SDE reads

$$dY_\tau = A(\tau)(Y_\tau - b_k) \, d\tau + \sqrt{\epsilon} \, dW_\tau.$$

Introduce the centered variable $\tilde{Y}_\tau := Y_\tau - b_k$:

$$d\tilde{Y}_\tau = A(\tau) \, \tilde{Y}_\tau \, d\tau + \sqrt{\epsilon} \, dW_\tau.$$

Fundamental solution. We seek a deterministic matrix $\Phi(\tau, s)$ such that $U_\tau = \Phi(\tau, 0)^{-1} \, \tilde{Y}_\tau$

eliminates the drift. Applying the product rule (no Itô correction since Φ is deterministic):

$$dU_\tau = \left[\frac{\partial \Phi^{-1}}{\partial \tau}(\tau, 0) + \Phi(\tau, 0)^{-1} A(\tau)\right] \tilde{Y}_\tau \, d\tau + \Phi(\tau, 0)^{-1} \sqrt{\epsilon} \, dW_\tau.$$

The drift vanishes when $\Phi(\tau, s)$ satisfies

$$\frac{\partial}{\partial \tau}\Phi(\tau, s) = A(\tau)\, \Phi(\tau, s), \qquad \Phi(s, s) = \mathbb{I}. \tag{2.5}$$

Commuting matrices. Since $A(\tau) = -\frac{1+\epsilon}{2}(\Sigma_k + (1-\tau)\mathbb{I})^{-1}$ and Σ_k is symmetric, $A(\tau)$ and $A(\tau')$ share a common eigenbasis for all τ, τ'. The solution is therefore

$$\Phi(\tau, s) = \exp\left(\int_s^\tau A(u)\, du\right) = \exp\left(-\frac{1+\epsilon}{2}\int_s^\tau (\Sigma_k + (1-u)\,\mathbb{I})^{-1} du\right).$$

Substituting $v = 1 - u$ (so $dv = -du$), the integral evaluates component-wise on each eigenvalue λ_i of Σ_k as

$$\int_s^\tau \frac{du}{\lambda_i + 1 - u} = \log \frac{\lambda_i + 1 - s}{\lambda_i + 1 - \tau}.$$

Therefore

$$\Phi(\tau, s) = \left[(\Sigma_k + (1-\tau)\,\mathbb{I})(\Sigma_k + (1-s)\,\mathbb{I})^{-1}\right]^{(1+\epsilon)/2}. \tag{2.6}$$

Integrating the SDE and undoing the substitution:

$$Y_\tau = b_k + \Phi(\tau, 0)\,(Y_0 - b_k) + \sqrt{\epsilon}\int_0^\tau \Phi(\tau, s)\, dW_s. \tag{2.7}$$

Evaluating at $\tau = 1$ (where $Y_0 \sim \mu = \mathcal{K}\pi^{(k)} = \mathcal{N}(b_k, \Sigma_k + \mathbb{I})$):

$$Y_1 = b_k + \Phi(1, 0)\,(Y_0 - b_k) + \sqrt{\epsilon}\int_0^1 \Phi(1, s)\, dW_s. \tag{2.8}$$

Since $Y_0 \sim \mathcal{N}(b, \Sigma)$, we can read off the distribution for Y_1

$$Y_1 = \mathcal{N}\left(b_k, \right) \tag{2.9}$$

2.3 Convergence

Fact 2 *For any symmetric matrix X and symmetric matrix B (s.t. $0 \prec B \prec \mathbb{I}$), $\|X - BXB\| \leq (1 - \lambda_{min}(B))^2 \|X\|$. Where $\|\cdot\| = \lambda_{max}(\cdot)$ is the spectral norm.*

Fact 3 (Gaussian AWGN Convergence) *Let $\eta = \min(\lambda_{min}(\Sigma_0), \lambda_{min}(\Sigma))$ and assume $\eta > 0$. The iteration in () converges as*

$$\|\Sigma_k - \Sigma\| \leq \left(1 - \left(\frac{\eta}{\eta + 1}\right)^{1+\epsilon}\right)^k \|\Sigma_0 - \Sigma\| \tag{2.10}$$

$$\|b_k - b\|^2 \leq \left(1 - \left(\frac{\eta}{\eta + 1}\right)^{1+\epsilon}\right)^k \|b_0 - b\|^2 \tag{2.11}$$

where $\|\Sigma\|$ is the spectral norm, and $\|b\|$ is the vector norm.

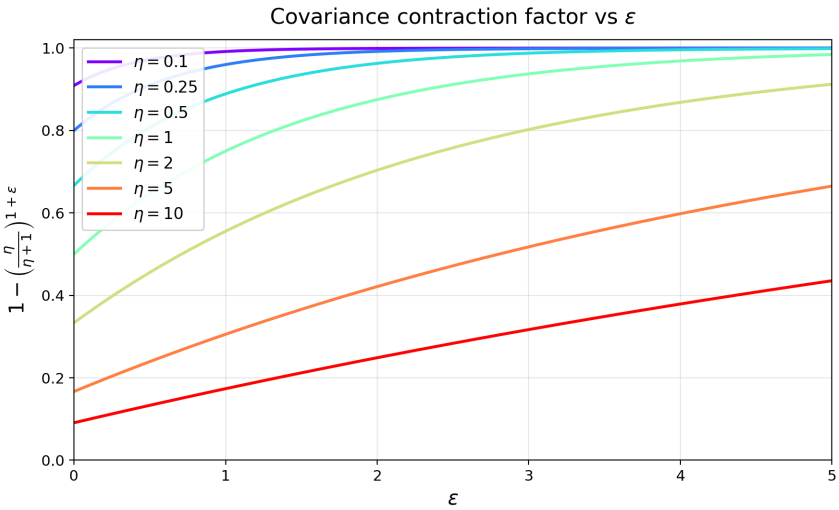


Figure 1: Convergence rate as a function of ϵ (noise level in backwards SDE). Convergence rate is quickest when $\epsilon = 0$, which also corresponds to using the ODE formalism.

2.4 Comparison to Expectation Maximization Algorithm

Since we've been making so many analogies to the EM algorithm, it'd be good to see if this can actually beat it.

$$\Sigma_{k+1} = \Sigma_k + M_k(\Sigma - \Sigma_k)M_k, \qquad M_k = \Sigma_k(\Sigma_k + \mathbb{I})^{-1} \tag{2.12}$$

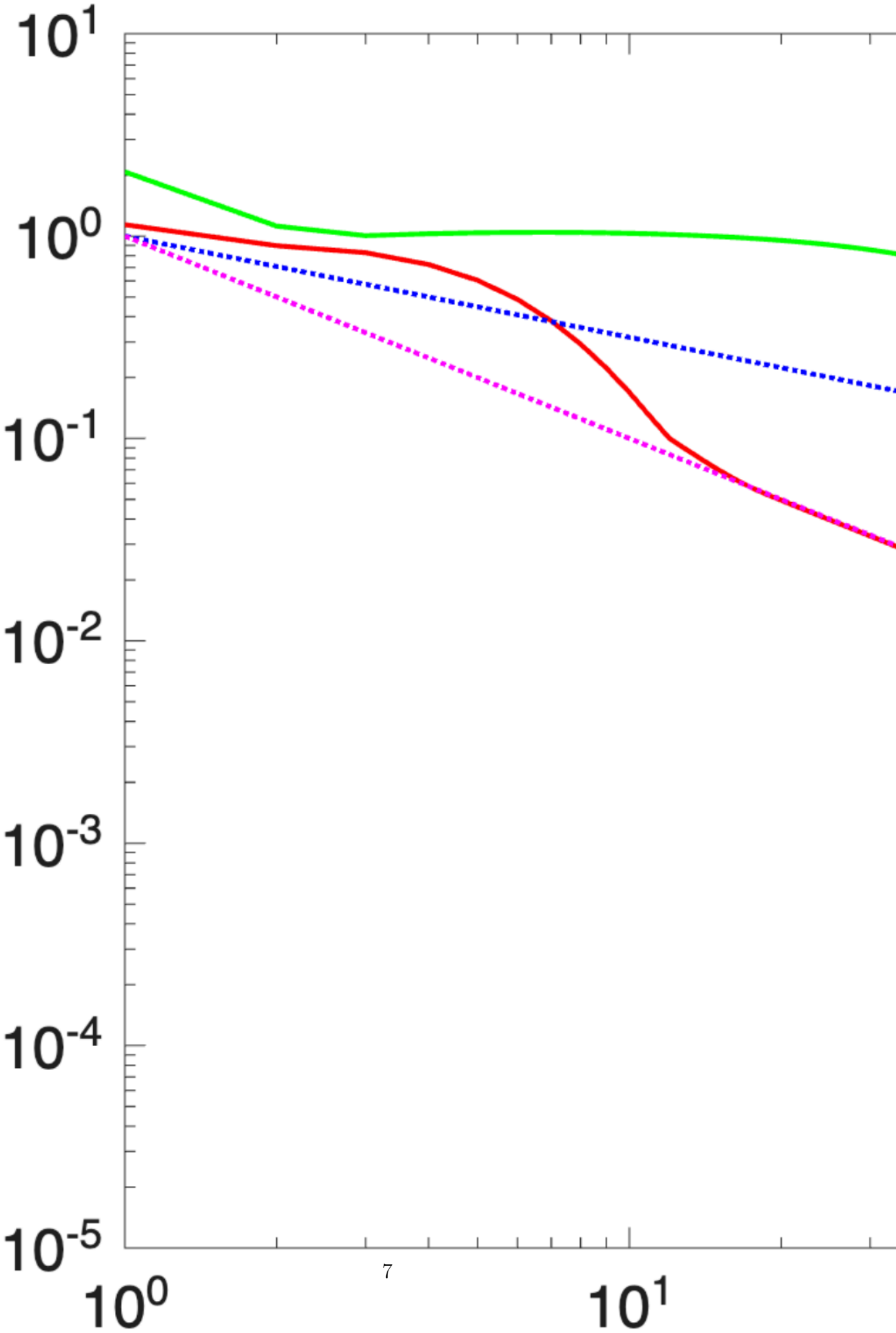
Notice this corresponds to $\epsilon = 1$ in our SCSI algorithm, which is suboptimal. So the new algorithm beats this out! In contraction rates, when $\eta \ll 1$, the $\epsilon = 0$ SCSI algorithm converges as $(1-\eta)^k$ while the $\epsilon = 1$ converges as $(1-\eta^2)^k$.

Some Guarantees

Fact 4 *We have $W_2^2(\pi, \pi^{(k)}) \leq R^k W_2^2(\pi, \pi^{(0)})$, and therefore we have exponential converges under $R < 1$.*

What's Next?

- Reward tilting for bayesian inverse problems?
- Inference time constraints?


10^1
10^0
10^{-1}
10^{-2}
10^{-3}
10^{-4}
10^{-5}
10^0
10^1